

INFORME PROYECTO DE AULA
DEFINICIÓN DEL PROBLEMA, FRONTERA DEL SISTEMA Y PROPUESTA
INICIAL DEL MODELO

Presentado por:

JUAN JOSÉ ÁLVAREZ MORA
DUVÁN VANEGAS PALACIO
RICHARD PALACIO QUINTERO
JEANS CARLOS ASPRILLA CÓRDOBA

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

FACULTAD DE INGENIERÍA

MODELOS Y SIMULACIÓN
GRUPO 002 ET 0199
ALEJANDRO VÁSQUEZ ECHEVERRI

INGENIERÍA DE SOFTWARE
MEDELLÍN, COLOMBIA
2025

Contenido

	Pág.
Introducción	6
1 Resumen	8
2 Definición del sistema	9
2.1 Problema a modelar	9
2.1.1 Clasificación del sistema	9
2.1.2 Frontera del sistema	10
2.1.3 Entradas	11
2.1.4 Salidas	12
2.1.5 Parámetros	13
2.1.6 Estado	14
2.1.7 Métricas de desempeño	14
2.1.8 Fuentes de datos	15
2.1.9 Verificación y Validación (V&V)	16
3 Implementación del Modelo	17
3.1 Traducción del modelo conceptual a computacional	17
3.2 Representación de incertidumbre	18
3.3 Supuestos adicionales	18
4 Experimentos y Resultados	19
4.1 Diseño experimental	19
4.2 Resultados preliminares	20
4.3 Análisis preliminar:	20
5 Código	21
6 Conclusiones Preliminares	35
Bibliografía	36

Lista de tablas

	Pág.
Tabla 1. Entradas del sistema	11
Tabla 2. Salidas del sistema	12
Tabla 3. Parámetros del sistema	13
Tabla 4. Estado del sistema	14
Tabla 5. Métricas de desempeño del sistema	14
Tabla 6. Incertidumbre en el sistema	18
Tabla 7. Tabla de resultados	20

Glosario

Agente de soporte: persona responsable de atender los tickets, cuya disponibilidad y gestión impactan directamente en la eficiencia del servicio.

Asignación de recursos: distribución estratégica de agentes y capacidades de atención para garantizar un servicio equilibrado y responder a la demanda de manera efectiva.

Eventos discretos: cambios puntuales en el sistema que ocurren en momentos específicos, como la llegada de una solicitud o la finalización de un servicio.

Estado del sistema: es la condición del helpdesk en un momento específico, considerando el número de tickets pendientes, agentes ocupados y tiempos de espera acumulados.

Helpdesk académico: es un sistema encargado de brindar soporte técnico a estudiantes y docentes, asegurando que sus solicitudes sean atendidas de manera eficiente y organizada.

Métricas de desempeño: indicadores que permiten evaluar la eficiencia del sistema, incluyendo tiempos de espera, duración de la atención, nivel de servicio y aprovechamiento de los agentes.

Modelo de simulación: es un esquema que reproduce el comportamiento del sistema, considerando la variabilidad en la llegada de tickets y los tiempos de atención para anticipar posibles escenarios.

Nivel de servicio: porcentaje de solicitudes atendidas dentro de un tiempo predefinido, indicador clave de la eficiencia del sistema.

Optimización: es el conjunto de acciones destinadas a mejorar la operación del helpdesk, en este caso, asegurando el uso eficiente de recursos y un servicio ágil.

Parámetros del modelo: variables que establecen las condiciones de la simulación, como número de agentes, horarios de atención, reglas de prioridad y capacidad de la cola de tickets.

Políticas de priorización: reglas que determinan el orden en que se atienden las solicitudes, por ejemplo, primero en entrar, primero en salir o por nivel de urgencia.

Recursos computacionales: elementos del software y hardware que permiten ejecutar la simulación y procesar la información del sistema.

Saturación: situación en la que la cantidad de solicitudes supera la capacidad del sistema, provocando demoras y afectando la calidad del servicio.

Simulación conceptual: representación simplificada del funcionamiento del helpdesk, utilizada para analizar su desempeño y probar distintos escenarios de gestión de recursos.

Ticket: registro de una solicitud de ayuda generada por un usuario dentro del sistema, que incluye información sobre el problema y su nivel de prioridad.

Validación y verificación: son los procesos que aseguran que el modelo refleja correctamente el funcionamiento real del helpdesk y que los resultados son confiables para la toma de decisiones.

Introducción

En los entornos académicos, garantizar un soporte técnico eficiente para estudiantes y docentes es fundamental para el buen desarrollo de las actividades educativas y administrativas. La atención oportuna a las solicitudes de ayuda contribuye a la satisfacción del usuario y al uso eficiente de los recursos, mientras que los tiempos de espera prolongados pueden generar retrasos, disminuir la calidad percibida del servicio y aumentar los costos operativos.

El presente informe propone un modelo de simulación de eventos discretos para analizar el flujo de solicitudes en un helpdesk académico, evaluando cómo la demanda de tickets y la disponibilidad de agentes impactan el desempeño del sistema. Este enfoque permite respaldar decisiones estratégicas sobre la asignación de personal y las políticas de priorización de solicitudes.

La construcción del modelo se fundamenta en teorías y metodologías reconocidas:

- Los principios de teoría de colas aportan bases analíticas para comprender el comportamiento de los tiempos de espera y la capacidad de servicio en sistemas de atención (Gross & Harris, 2008).
- La simulación de eventos discretos proporciona herramientas para modelar procesos dinámicos y estocásticos, como la llegada y atención de tickets (Banks et al., 2010).
- Las técnicas de análisis y validación de simulaciones permiten interpretar los resultados y diseñar experimentos que reflejan escenarios operativos reales (Law, 2015).
- La consideración de tasa de llegadas variables ayuda a capturar la fluctuación de la demanda en distintos momentos, característica común en sistemas de soporte académico (Whitt, 1993).
- Finalmente, los métodos de control estadístico de calidad facilitan la evaluación de métricas de desempeño y la verificación de la precisión de los datos utilizados (Montgomery, 2020).

De esta manera, el modelo integra teoría de colas, simulación de eventos discretos y análisis estadístico para representar de forma precisa la operación del helpdesk académico.

Esto permite explorar diferentes escenarios, anticipar posibles cuellos de botella y evaluar indicadores clave, como tiempos promedio de espera, duración de la atención, nivel de servicio y utilización de agentes. En consecuencia, el modelo se convierte en una herramienta estratégica para planificar, optimizar y mejorar continuamente el servicio de soporte académico.

1. Resumen

El presente proyecto aborda la optimización de la asignación de recursos en un sistema de soporte técnico académico (helpdesk) que gestiona solicitudes de estudiantes y docentes. La investigación se centra en analizar la interacción entre la demanda de tickets y la disponibilidad de agentes, con el fin de mejorar la eficiencia operativa y la calidad del servicio. Mediante la construcción de un modelo de simulación conceptual, se evalúan métricas clave de desempeño como tiempos promedio de espera, tiempos de atención, utilización de agentes y nivel de servicio, permitiendo identificar escenarios de saturación y oportunidades de mejora en la asignación de personal.

El enfoque adoptado facilita la exploración de estrategias de priorización y distribución de recursos, ofreciendo soporte a la toma de decisiones para garantizar un servicio ágil y equilibrado. Este modelo constituye una herramienta valiosa para planificar la capacidad del helpdesk, anticipar cuellos de botella y optimizar los procesos internos, contribuyendo al cumplimiento de estándares de eficiencia y satisfacción de los usuarios. Los resultados obtenidos proporcionan un marco sólido para orientar intervenciones estratégicas, mejorar la experiencia del usuario y maximizar el aprovechamiento de los recursos disponibles.

2. Definición del sistema

2.1 Problema a modelar

El sistema a modelar corresponde a un helpdesk académico, encargado de gestionar solicitudes de estudiantes y docentes. Cada solicitud se registra como un ticket, que es atendido por un agente según reglas de prioridad establecidas por la institución.

El problema central consiste en evaluar cómo varían los tiempos de espera y de atención de los usuarios según dos factores principales: la demanda de tickets (número y frecuencia de llegada) y la capacidad del sistema (número de agentes disponibles y duración de sus turnos). La simulación permitirá analizar distintos escenarios y apoyar la toma de decisiones sobre asignación de personal y políticas de priorización.

Hipótesis del modelo:

- Cada ticket requiere la atención de un solo agente.
- Todos los agentes tienen la misma capacidad de atención y trabajan en turnos fijos.
- Las llegadas de tickets y los tiempos de servicio siguen distribuciones probabilísticas (Exponencial y Normal, respectivamente).

Horizonte de tiempo y granularidad:

- La simulación abarca una semana operativa, considerando jornadas de 8:00 a 18:00.
- La granularidad temporal se establece en minutos, lo que permite medir con precisión los tiempos de espera y servicio de cada ticket.

2.1.1 Clasificación del sistema

El sistema de soporte técnico académico se clasifica de la siguiente manera:

- **Estático / Dinámico:** Dinámico, dado que el estado del sistema cambia constantemente a lo largo del tiempo en función de la llegada de nuevos tickets y la atención que los agentes prestan.

- **Determinista / Estocástico:** Estocástico, ya que tanto la generación de tickets como los tiempos de servicio presentan variabilidad e incertidumbre, lo que impide predecir de manera exacta el comportamiento del sistema.
- **Continuo / Discreto / Híbrido:** Discreto, porque los cambios en el sistema ocurren en instantes específicos asociados a eventos puntuales, como la llegada de un ticket, el inicio o finalización de su atención.
- **Abierto / Cerrado:** Abierto, dado que los usuarios pueden generar solicitudes en cualquier momento, y el sistema debe adaptarse a estas entradas externas de forma continua.

2.1.2 Frontera del sistema

Se define la frontera del helpdesk académico para delimitar claramente qué elementos que forman parte del sistema y cuáles se consideran externos:

Dentro del sistema

- **Tickets:** solicitudes de soporte generadas por estudiantes o docentes que ingresan al sistema para ser atendidas.
- **Agentes de soporte:** personal encargado de gestionar los tickets, cuya disponibilidad y eficiencia impactan directamente en el desempeño del helpdesk.
- **Reglas de asignación:** políticas que determinan el orden de atención de los tickets, ya sea por prioridad, por orden de llegada o por otras estrategias definidas por la institución.
- **Cola de espera:** estructura que organiza los tickets pendientes hasta que sean asignados a un agente disponible.

Fuera del sistema

- **Usuarios externos:** estudiantes y docentes que generan las solicitudes de soporte, pero cuya gestión directa queda fuera del modelo de simulación.
- **Infraestructura tecnológica:** servidores, redes y plataformas de software que facilitan la comunicación, pero que no se modelan explícitamente en la simulación.

2.1.3 Entradas

Las entradas del sistema representan los factores que afectan directamente la operación del helpdesk académico y que se deben considerar al modelar su comportamiento. Cada entrada se define como una variable con unidad y tipo de dato:

Tabla 1.
Entradas del sistema

Variable	Descripción	Unidad	Tipo	Observación / Fuente
λ_{tickets}	Tasa promedio de llegada de tickets al helpdesk	tickets/minuto	numérica continua	Puede variar según hora del día, día de la semana o eventos externos
P_prioridad	Distribución de prioridades de los tickets (alta, media, baja)	%	categoría	Determina el orden de atención según política de priorización
N_agentes	Número de agentes disponibles en un turno	agentes	numérica discreta	Define la capacidad máxima de atención simultánea
Horario	Intervalo de operación del helpdesk	horas	numérica continua	Horas activas de los agentes; puede incluir cambios de turno

Fuente: Elaboración propia del equipo.

2.1.4 Salidas

Las salidas del sistema representan los resultados o métricas de desempeño del helpdesk académico. Cada salida se define como una variable medible que permite evaluar la eficiencia y calidad del servicio:

Tabla 2.
Salidas del sistema

Variable	Descripción	Unidad	Tipo	Observación / Fórmula
T_espera	Tiempo promedio que un ticket permanece en cola antes de ser atendido	minutos	numérica continua	$T_espera = \frac{\sum(t_inicio_atencion - t_llegada)}{N_tickets}$
T_sistema	Tiempo total promedio que un ticket tarda desde su creación hasta su resolución	minutos	numérica continua	$T_sistema = \frac{\sum(t_resolucion - t_llegada)}{N_tickets}$
U_agentes	Porcentaje de tiempo que los agentes están ocupados atendiendo tickets respecto al tiempo total disponible	%	numérica continua	$U_agentes = \frac{tiempo_ocupado_total}{tiempo_disponible_total} * 100$
N_cola	Número promedio de tickets esperando atención	tickets	numérica continua	Promedio temporal de tickets en cola durante la simulación
NS	Nivel de servicio: porcentaje de tickets atendidos dentro de un tiempo máximo definido (ej. <10 min)	%	numérica continua	$NS = \frac{N_tickets_atendidos_a_tiempo}{N_tickets_totales} * 100$

Fuente: Elaboración propia del equipo.

2.1.5 Parámetros

Tabla 3.

Parámetros del sistema

Parámetro	Descripción	Unidad	Tipo	Observación / Fórmula
Pol_asignacion	Regla que determina el orden de atención de los tickets	N/A	categorica	FIFO, prioridad alta primero, enrutamiento por habilidades del agente
Cap_cola	Capacidad máxima de tickets en cola	tickets	numérica discreta	Si se supera, los tickets nuevos pueden ser rechazados o redirigidos
Dur_turno	Duración de los turnos de los agentes	horas	numérica continua	Define el tiempo que cada agente permanece disponible para atención
Tiempo_servicio	Tiempo promedio de atención por ticket	minutos	numérica continua	Se puede modelar con distribución probabilística (Normal, Gamma)
Tiempo_llegada	Tiempo entre llegadas de tickets	minutos	numérica continua	Se puede modelar con distribución probabilística (Exponencial)

Fuente: Elaboración propia del equipo.

2.1.7 Estado

Tabla 4.
Estado del sistema

Variable	Descripción	Unidad / Tipo	Fuente / Nota
Tickets en cola	Número de tickets pendientes de atención	Entero	Sistema de helpdesk
Estado de agentes	Condición del agente: ocupado o libre	Categorico	Observación del sistema
Tiempo acumulado espera	Suma de los tiempos de espera de todos los tickets	Minutos	Cálculo a partir de timestamps
Tickets atendidos	Número de tickets resueltos en un período específico	Entero	Sistema de helpdesk

Fuente: Elaboración propia del equipo.

2.1.7 Métricas de desempeño

Tabla 5.
Métricas de desempeño del sistema

Métrica	Descripción	Unidad / Tipo	Valor de referencia (si aplica)
Tiempo promedio de espera	Promedio de minutos que un ticket espera en cola antes de ser atendido	Minutos	10 min (nivel de servicio)
Tiempo promedio en sistema	Promedio desde la llegada hasta la resolución del ticket	Minutos	N/A
Utilización de agentes	Porcentaje de tiempo que los agentes están ocupados atendiendo tickets	%	80–90% recomendado
Número promedio tickets cola	Promedio de tickets esperando atención	Entero	N/A
Nivel de servicio	Porcentaje de tickets atendidos dentro del tiempo definido	%	> 80% recomendado

Fuente: Elaboración propia del equipo.

2.1.8 Fuentes de datos

Disponibles:

- Registros históricos del helpdesk académico: incluyen información detallada sobre cada ticket, como los timestamps de creación, asignación y resolución; nivel de prioridad; canal de contacto; agente asignado; y estado final del ticket.

Supuestos iniciales:

- Las llegadas de tickets y los tiempos de servicio se moldearán inicialmente mediante distribuciones sencillas.
- Todos los supuestos serán validados estadísticamente y ajustados según la evidencia empírica de los datos históricos.

Plan de recolección de datos:

- Recopilar al menos un mes de registros operativos para capturar variaciones horarias y semanales.

Campos mínimos a incluir:

- ticket_id, timestamp_llegada, timestamp_asignacion, timestamp_resolucion, prioridad, canal, agente_id, categoria, estado_final

Pasos de limpieza y preparación:

- Unificar zonas horarias.
- Eliminar registros duplicados.
- Corregir o imputar timestamps faltantes.

2.1.9 Verificación y Validación (V&V)

Verificación:

- Asegurar que la implementación del modelo refleja correctamente las reglas definidas para el helpdesk académico.
- Realizar pruebas con escenarios controlados de tickets para comprobar que los procesos de llegada, cola y atención funcionan según lo esperado.

Validación:

- Comparar métricas de desempeño simuladas con datos históricos del sistema real, por ejemplo: tiempo promedio de espera y nivel de servicio.
- Ajustar parámetros del modelo según las diferencias observadas para mejorar la fidelidad.

Pruebas de sensibilidad:

- Variar parámetros clave, como número de agentes o políticas de priorización, para evaluar cómo afectan los resultados.
- Validar los resultados con expertos del dominio (personal del helpdesk académico) para asegurar consistencia con la operación real.

Objetivo:

- Garantizar que el modelo conceptual y computacional representa fielmente el sistema, que los datos utilizados son confiables, y que las decisiones basadas en la simulación son fundamentadas.

3. Implementación del Modelo

3.1 Traducción del modelo conceptual a computacional

El modelo conceptual desarrollado en la fase anterior se transformó en un modelo de simulación funcional utilizando Python con SimPy, aunque puede adaptarse a otras librerías de simulación discreta. Los elementos principales se representan de la siguiente manera:

Entidades: Tickets generados por los usuarios, caracterizados por:

- ID del ticket
- Prioridad
- Timestamp de llegada
- Timestamp de asignación
- Timestamp de resolución

Recursos: Agentes de soporte, modelados como recursos limitados que atienden los tickets en función de su disponibilidad.

Actividades del sistema:

- Llegada de tickets según distribuciones de tiempo.
- Gestión de la cola de espera, que puede ser FIFO (primero en entrar, primero en salir) o priorizada según nivel de urgencia.
- Asignación de tickets a agentes y atención con tiempos de servicio probabilísticos, simulando la variabilidad natural de la operación.

3.2 Representación de incertidumbre

Se incorporó variabilidad mediante distribuciones de probabilidad, que permiten reflejar la naturaleza estocástica del sistema:

Tabla 6.
Incetidumbre en el sistema

Elemento	Distribución	Justificación
Tiempo entre llegadas de tickets	Exponencial (λ ajustado por hora del día)	Común en colas tipo M/M/c; validado con registros históricos
Tiempo de servicio por agente	Normal truncada o Gamma	Refleja la variabilidad natural en la atención; asegura valores positivos

Fuente: Elaboración propia del equipo.

Se implementaron funciones de muestreo para cada distribución, garantizando independencia de eventos y estocasticidad del sistema.

3.3 Supuestos adicionales

- No se considera abandono de tickets por parte de los usuarios.
- Cada ticket requiere un solo agente para completarse.
- La duración de turnos es constante y todos los agentes tienen la misma capacidad.

4. Experimentos y Resultados

4.1 Diseño experimental

El diseño experimental busca evaluar el desempeño del helpdesk académico bajo diferentes configuraciones de agentes y políticas de priorización. Los parámetros considerados son los siguientes:

- **Número de corridas por escenario:** 30, para capturar la variabilidad natural del sistema.
- **Duración de la simulación:** 1 semana operativa, considerando horarios de atención de 8:00 a 18:00.
- **Escenarios evaluados:**
 - **Número de agentes:** 3, 5 y 7.
 - **Políticas de priorización:** FIFO (primero en entrar, primero en salir) y prioridad alta primero.
- **Métricas de desempeño:**
 - **Tiempo promedio de espera (minutos):** duración media que un ticket permanece en cola antes de ser atendido.
 - **Tiempo promedio en el sistema (minutos):** tiempo total desde la llegada hasta la resolución de un ticket.
 - **Utilización de agentes (%):** porcentaje del tiempo que los agentes están ocupados atendiendo tickets.
 - **Número promedio de tickets en cola:** indicador de congestión del sistema.
 - **Nivel de servicio (%):** porcentaje de tickets atendidos en menos de 10 minutos.

4.2 Resultados preliminares

Los resultados obtenidos se resumen en la siguiente tabla, expresados como promedios de 30 corridas por escenario:

Tabla 7.

Tabla de resultados

Escenario	Tiempo espera (min)	Tiempo en sistema (min)	Utilización agentes (%)	Nivel de servicio (%)
3 agentes	3.08	15.11	67.41	88.49
5 agentes	0.13	12.08	38.50	99.76
7 agentes	0.01	11.93	27.85	100.0

Fuente: Elaboración propia del equipo.

4.3 Análisis preliminar

- **Impacto del número de agentes:** El incremento de agentes de 3 a 7 reduce drásticamente el tiempo promedio de espera, pasando de 3.08 minutos a menos de 0.01 minutos, lo que evidencia un efecto directo sobre la congestión del sistema y la experiencia del usuario..
- **Efecto de la congestión:** El nivel de servicio mejora de 88.5% con 3 agentes a 100% con 7 agentes, garantizando que prácticamente todos los tickets sean atendidos en menos de 10 minutos. Sin embargo, el aumento de agentes también reduce la utilización promedio de 67% a 28%, indicando capacidad ociosa en los escenarios con mayor dotación.
- **Balance capacidad-demanda:** El escenario con 5 agentes parece representar un punto de equilibrio: mantiene una utilización moderada (38%) y un nivel de servicio superior al 99%, optimizando los recursos sin sobreasignación de personal.

5. Código

```
# -----

# SIMULACIÓN DE HELPDESK ACADÉMICO (DES)

# -----

# Este script modela un helpdesk académico mediante una simulación de eventos
discretos.

# Se evalúa el desempeño del sistema bajo distintas cantidades de agentes de
soporte.

!pip install simpy

import simpy
import random
import math
import numpy as np
import pandas as pd
from typing import List, Optional, Dict

# -----

# PARÁMETROS GLOBALES

# -----

class glo:

    """

    Clase con parámetros globales de la simulación.

    Atributos:

        mean_iat (float): Tiempo promedio entre llegadas (minutos).
```

```

    mean_service (float): Tiempo promedio de servicio por ticket (minutos).

    cv_service (float): Coeficiente de variación del tiempo de servicio.

    sim_duration (float): Duración total de la simulación (minutos).

    sla_wait (float): Tiempo máximo de espera permitido por SLA.

    base_seed (int): Semilla base para la generación de números aleatorios.

    """

    mean_iat = 6.0

    mean_service = 12.0

    cv_service = 0.6

    sim_duration = 480.0

    sla_wait = 10.0

    base_seed = 12345

# -----
# FUNCIONES AUXILIARES
# -----

def lognorm_mu_sigma_from_mean_cv(mean: float, cv: float):

    """

    Calcula los parámetros mu y sigma de una distribución lognormal

    a partir de su media y coeficiente de variación.

    Args:

        mean (float): Media deseada de la distribución.

        cv (float): Coeficiente de variación (desviación estándar / media).

    Returns:

```

```

        tuple: (mu, sigma) parámetros para np.random.lognormal.

    """

    sigma = math.sqrt(math.log(1 + cv**2))

    mu = math.log(mean) - 0.5 * sigma**2

    return mu, sigma

# Cálculo de los parámetros lognormales globales
MU_LN, SIGMA_LN = lognorm_mu_sigma_from_mean_cv(glo.mean_service, glo.cv_service)

# -----
# CLASE JOB (TICKET)
# -----

class Job:

    """

    Representa una solicitud (ticket) en el sistema de helpdesk.

    Atributos:

        id (int): Identificador único del ticket.

        arrival (float): Tiempo de llegada al sistema.

        start (float): Tiempo de inicio de atención.

        end (float): Tiempo de finalización de la atención.

    """

    def __init__(self, job_id: int):

        self.id = job_id

```

```

self.arrival: Optional[float] = None

self.start: Optional[float] = None

self.end: Optional[float] = None


@property
def wait(self) -> Optional[float]:
    """
    Calcula el tiempo de espera en cola del ticket.

    Returns:
        float | None: Tiempo entre la llegada y el inicio de atención.
    """
    if self.arrival is None or self.start is None:
        return None

    return self.start - self.arrival


@property
def total_time(self) -> Optional[float]:
    """
    Calcula el tiempo total del ticket en el sistema (espera + servicio).

    Returns:
        float | None: Tiempo entre la llegada y la finalización.
    """
    if self.arrival is None or self.end is None:
        return None

    return self.end - self.arrival

```



```

# -----
# CLASE MODEL (SIMULACIÓN DE UNA RÉPLICA)
# -----

class Model:

    """

    Ejecuta una simulación de una réplica del helpdesk con un número fijo de
    agentes.

    Args:

        run_number (int): Número de réplica (para diferenciar corridas).

        n_agents (int): Cantidad de agentes de soporte en el sistema.

        seed (int, opcional): Semilla aleatoria para reproducibilidad.

    Atributos:

        env (simpy.Environment): Entorno de simulación.

        server (simpy.Resource): Recurso que representa a los agentes.

        jobs (List[Job]): Lista con los tickets procesados.

        total_service_time (float): Suma de todos los tiempos de servicio.

        completed (int): Número de tickets atendidos completamente.

    """

    def __init__(self, run_number: int, n_agents: int, seed: Optional[int] = None):

        self.run_number = run_number

        self.n_agents = n_agents

```

```

        self.seed = seed if seed is not None else glo.base_seed + run_number +
n_agents * 1000

        random.seed(self.seed)

        np.random.seed(self.seed + 1)

        self.env = simpy.Environment()

        self.server = simpy.Resource(self.env, capacity=n_agents)

        self.job_counter = 0

        self.jobs: List[Job] = []

        self.total_service_time = 0.0

        self.completed = 0

    def sample_service_time(self) -> float:

        """

        Genera un tiempo de servicio aleatorio con distribución lognormal.

        Returns:

            float: Tiempo de servicio en minutos.

        """

        return float(np.random.lognormal(MU_LN, SIGMA_LN))

    def generator_job_arrivals(self):

        """

        Genera la llegada de tickets al sistema siguiendo una distribución
exponencial.

```

```

Inicia el proceso de atención de cada ticket creado.

"""

while True:

    self.job_counter += 1

    job = Job(self.job_counter)

    job.arrival = self.env.now

    self.jobs.append(job)

    self.env.process(self.process_job(job))

    # Tiempo entre llegadas

    iat = random.expovariate(1.0 / glo.mean_iat)

    yield self.env.timeout(iat)

def process_job(self, job: Job):

    """

    Proceso que representa la atención de un ticket por un agente.

    Args:

        job (Job): Ticket a procesar.

    """

    with self.server.request() as req:

        yield req # Espera hasta que haya un agente libre

        job.start = self.env.now

        s = self.sample_service_time()

        self.total_service_time += s

        yield self.env.timeout(s)

        job.end = self.env.now

```

```

        self.completed += 1

def run(self):

    """
    Ejecuta la simulación completa hasta que se cumple el tiempo total definido.
    Al finalizar, genera un DataFrame con los datos de los tickets atendidos.
    """

    self.env.process(self.generator_job_arrivals())

    self.env.run(until=glo.sim_duration)

    # Construcción del DataFrame de resultados

    records = []

    for j in self.jobs:

        rec = {

            "id": j.id,

            "arrival": j.arrival,

            "start": j.start,

            "end": j.end,

            "wait": j.wait,

            "total": j.total_time

        }

        records.append(rec)

    self.df_jobs = pd.DataFrame(records)

# -----
# CLASE TRIAL (EXPERIMENTOS)

```

```

# -----

class Trial:

    """

    Ejecuta múltiples réplicas del modelo para distintos números de agentes.

    Calcula métricas promedio y genera un resumen estadístico por escenario.

    Args:

        n_agents_list (List[int]): Lista con las cantidades de agentes a probar.

        n_runs (int): Número de réplicas por escenario.

    Atributos:

        results (List[Dict]): Resultados detallados y resumen por escenario.

    """

    def __init__(self, n_agents_list: List[int], n_runs: int = 30):

        self.n_agents_list = n_agents_list

        self.n_runs = n_runs

        self.results: List[Dict] = []

    def run(self):

        """

        Ejecuta todas las réplicas de simulación para cada configuración de agentes.

        Calcula métricas como tiempo promedio de espera, percentiles, throughput y
utilización.

        """

        for n in self.n_agents_list:

            per_run = []

```

```

for r in range(self.n_runs):

    seed = glo.base_seed + n * 10000 + r

    m = Model(run_number=r, n_agents=n, seed=seed)

    m.run()

    dfj = m.df_jobs

    # Métricas por réplica

    total_arrivals = len(m.jobs)

    completed = m.completed

    wait_vals = dfj['wait'].dropna().values

    mean_wait = float(np.mean(wait_vals)) if len(wait_vals) > 0 else
float('nan')

    p90_wait = float(np.percentile(wait_vals, 90)) if len(wait_vals) > 0
else float('nan')

    total_vals = dfj['total'].dropna().values

    mean_total = float(np.mean(total_vals)) if len(total_vals) > 0 else
float('nan')

    throughput_h = completed / (glo.sim_duration / 60.0)

    n_meet = int((dfj['wait'] < glo.sla_wait).sum()) if len(dfj) > 0
else 0

    pct_meet = 100.0 * n_meet / total_arrivals if total_arrivals > 0
else float('nan')

    util = m.total_service_time / (n * glo.sim_duration) if n > 0 else
float('nan')

    per_run.append({

        "n_agents": n,

```

```

        "run": r,

        "total_arrivals": total_arrivals,

        "completed": completed,

        "mean_wait": mean_wait,

        "p90_wait": p90_wait,

        "mean_total": mean_total,

        "throughput_h": throughput_h,

        "pct_sla": pct_meet,

        "utilization": util

    })

df_perrun = pd.DataFrame(per_run)

def ci95(arr):

    """

    Calcula el intervalo de confianza del 95% para un arreglo de
valores.

    Args:

        arr (array-like): Conjunto de valores numéricos.

    Returns:

        tuple: (límite_inferior, límite_superior)

    """

    arr = np.array(arr)

    arr = arr[~np.isnan(arr)]

    if len(arr) == 0:

```

```

        return (np.nan, np.nan)

    m = arr.mean()

    se = arr.std(ddof=1) / math.sqrt(len(arr))

    delta = 1.96 * se

    return (m - delta, m + delta)

# Resumen de las métricas promedio por escenario

summary = {

    "n_agents": n,

    "reps": len(df_perrun),

    "mean_wait_mean": df_perrun['mean_wait'].mean(),

    "mean_wait_ci95": ci95(df_perrun['mean_wait']),

    "p90_wait_mean": df_perrun['p90_wait'].mean(),

    "mean_total_mean": df_perrun['mean_total'].mean(),

    "throughput_h_mean": df_perrun['throughput_h'].mean(),

    "pct_sla_mean": df_perrun['pct_sla'].mean(),

    "utilization_mean": df_perrun['utilization'].mean()

}

self.results.append({

    "n_agents": n,

    "per_run": df_perrun,

    "summary": summary

})

def to_summary_df(self):

    """

```


Convierte los resultados resumidos de todas las configuraciones
en un DataFrame consolidado y ordenado por número de agentes.

Returns:

pd.DataFrame: Tabla con las métricas promedio y sus intervalos.

"""

rows = []

for res in self.results:

s = res['summary']

rows.append({

"n_agents": s['n_agents'],

"reps": s['reps'],

"mean_wait_mean": s['mean_wait_mean'],

"mean_wait_ci95_low": s['mean_wait_ci95'][0],

"mean_wait_ci95_high": s['mean_wait_ci95'][1],

"p90_wait_mean": s['p90_wait_mean'],

"mean_total_mean": s['mean_total_mean'],

"throughput_h_mean": s['throughput_h_mean'],

"pct_sla_mean": s['pct_sla_mean'],

"utilization_mean": s['utilization_mean']

})

return pd.DataFrame(rows).sort_values("n_agents")

EJECUCIÓN DE SIMULACIÓN

```
if __name__ == "__main__":  
  
    """  
  
    Punto de entrada principal.  
  
    Define los escenarios (número de agentes) y ejecuta los experimentos.  
  
    Finalmente, imprime el resumen de resultados y lo exporta a CSV.  
  
    """  
  
    trial = Trial(n_agents_list=[3, 5, 7], n_runs=30)  
  
    trial.run()  
  
    df_summary = trial.to_summary_df()  
  
    print(df_summary.to_string(index=False))  
  
    df_summary.to_csv("resumen_simulacion_helpdesk.csv", index=False)
```

6. Conclusiones Preliminares

La simulación desarrollada proporciona una herramienta valiosa para cuantificar el impacto de la variabilidad en la llegada de tickets y en los tiempos de atención sobre las métricas de desempeño del helpdesk académico. Los resultados sugieren que:

- Incrementar los agentes mejora significativamente los tiempos de espera y el cumplimiento del nivel de servicio.
- Sin embargo, a partir de cierto punto (más de 5 agentes), la mejora marginal es mínima y la utilización de recursos cae considerablemente.
- El modelo permite identificar escenarios de saturación y puntos de equilibrio operativos útiles para la toma de decisiones.

Limitaciones:

- **Abandono de tickets:** No se consideró la posibilidad de que los usuarios cancelen sus solicitudes antes de ser atendidos.
- **Tickets críticos multi-agente:** No se modelaron situaciones donde un ticket requiera la atención simultánea de varios agentes.
- **Distribuciones iniciales:** Las distribuciones de llegada y servicio empleadas se basan en supuestos iniciales que requieren validación con datos históricos más completos.

Próximos pasos:

- **Refinamiento de distribuciones:** Ajustar las distribuciones de probabilidad utilizando registros históricos reales para mejorar la representatividad del modelo.
- **Incorporación de métricas adicionales:** Incluir indicadores de satisfacción del usuario y análisis de sensibilidad más profundo para evaluar la robustez del sistema.
- **Documentación y depuración:** Revisar y optimizar el código del modelo para garantizar su correcta implementación y facilitar futuras extensiones.

Bibliografia

- Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). Discrete-event system simulation (5th ed.). Pearson.
- Gross, D., & Harris, C. M. (2008). *Fundamentals of queueing theory* (4th ed.). Wiley.
- Law, A. M. (2015). *Simulation modeling and analysis* (5th ed.). McGraw-Hill Education.
- Montgomery, D. C. (2020). *Introduction to statistical quality control* (9th ed.). Wiley.
- Whitt, W. (1993). Approximations for time-varying queues. *Management Science*, 39(4), 525–537.